



Universidad de Oviedo
Grado en Física

Introducción a la Física Computacional

Curso 2023–24

Representaciones gráficas en 2D y 3D

Índice general

8. Representaciones gráficas en 2D y 3D	2
8.1. Objetivo	2
8.2. ¿Cómo funciona Matplotlib?	2
8.3. Gráficas en 2D	3
8.3.1. Cómo usar <code>pyplot</code>	3
8.3.2. Salida interactiva o a fichero	3
8.3.3. Dibujando un conjunto de puntos	3
8.3.4. Dibujo de funciones	6
8.3.5. Etiquetando las curvas	8
8.3.6. Control de los ejes	9
8.3.7. Guardando la figura	10
8.3.8. Controlando propiedades de las líneas	11
8.3.9. Varias subfiguras	11
8.3.10. Otro tipos de gráficas	12
8.3.11. Anotando figuras	15
8.4. Gráficas en 3D	17
8.4.1. Usando <code>mplot3d</code>	17
8.4.2. Dibujo de curvas con <code>mplot3d</code>	18
8.4.3. Superficies con <code>mplot3d</code>	19

Práctica 8

Representaciones gráficas en 2D y 3D

8.1. Objetivo

Familiarizarse con los módulos de representaciones gráficas de [Python](#) incluidos en el paquete [Matplotlib](#).

8.2. ¿Cómo funciona Matplotlib?

Existen diversos paquetes de módulos para representar gráficas en 2D y 3D. Usaremos el paquete [Matplotlib](#), considerado como una especie de 'estándar'.

Matplotlib tiene el concepto de 'figura actual', también con el 'área de dibujo actual' y 'ejes actuales', pudiendo cambiarse en cada momento cualquiera de ellos.

Una figura puede contener subfiguras (diferentes zonas de dibujo), y se pueden anotar con títulos, títulos de ejes, etc.

El módulo para las gráficas bidimensionales es [matplotlib.pyplot](#), mientras que para las gráficas tridimensionales es [matplotlib.mplot3d](#).

Existen más módulos en el paquete [matplotlib](#).

Hay más información disponible en <http://matplotlib.org>.

8.3. Gráficas en 2D

Para representaciones en 2D usaremos el módulo `Matplotlib.pyplot`, que contiene una colección de funciones que son similares a las usadas en Matlab[®].

Se puede encontrar una referencia completa de la funcionalidad disponible en `pyplot` en la siguiente dirección: https://matplotlib.org/stable/api/pyplot_summary.html

8.3.1. Cómo usar `pyplot`

Para evitar “contaminaciones” del espacio global de variables y funciones lo importaremos así:

```
import matplotlib.pyplot as plt
```

El primer comando gráfico que empleemos crea una figura a la que irán los elementos gráficos que se vayan añadiendo. Para crear expresamente una nueva figura usaremos

```
plt.figure()
```

8.3.2. Salida interactiva o a fichero

Los comandos gráficos no se irán mostrando en pantalla hasta que se utilice el comando:

```
plt.show()
```

Sin embargo es posible que no nos interese mostrar los resultados en pantalla, sino en un fichero gráfico con el comando:

```
plt.savefig()
```

8.3.3. Dibujando un conjunto de puntos

Una de las representaciones gráficas más habituales en física es aquella en el que representamos en el plano un conjunto de puntos (x_i, y_i) correspondientes a la medida de una magnitud (Y) que consideramos dependiente de un parámetro (X). La función de `pyplot` para dibujar una gráfica de este tipo es `plot()`. Admite un conjunto amplio de parámetros¹ que permiten especificar numerosos aspectos estéticos (y otros), pero en su versión más sencilla tiene la forma:

```
plt.plot(X,Y, formato)
```

donde:

- X e Y son colecciones (tuplas, listas, arrays,...) con las coordenadas que definen los puntos (x_i, y_i) de la gráfica

¹Detalles en https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html

- **formato** (opcional) es una cadena de caracteres que combinando caracteres simple describe el *formato* de la gráfica: tipo de línea (continua, punteada,...), de marcador (círculo, cuadrado,...) y color. Si no se especifica este parámetro se dibujará una línea continua azul uniendo los puntos.

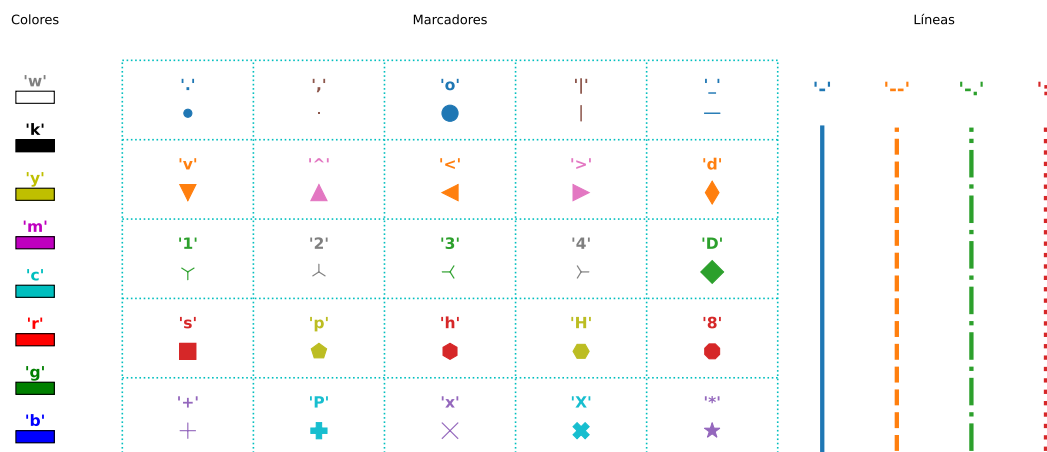


Figura 8.1: Códigos de colores, marcadores y líneas en Matplotlib

Ejemplo 8.3.1. Un ejemplo simple ([plt1.py](#))

```
'''
Ejemplo simple de figura 2D con pyplot
'''
import matplotlib.pyplot as plt

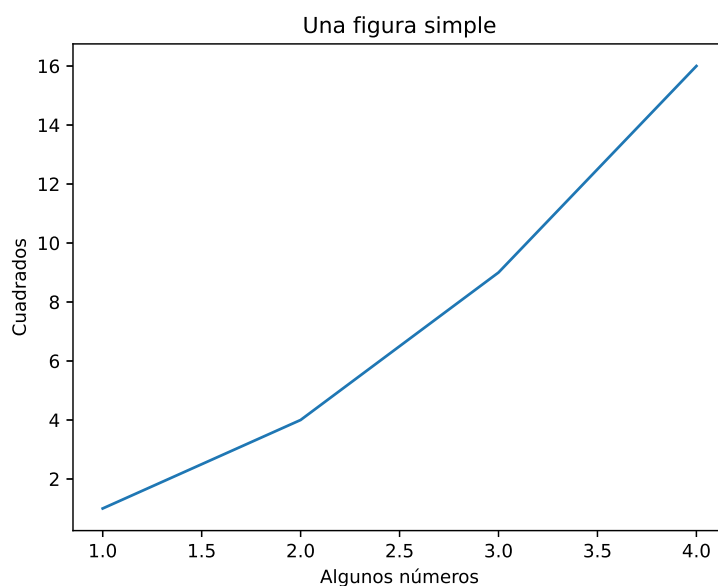
# Creamos la gráfica
plt.plot([1,2,3,4], [1,4,9,16])

# Y la adornamos
plt.title('Una figura simple') # Título de la figura
plt.xlabel('Algunos números')  # Etiqueta eje X
plt.ylabel('Cuadrados')        # Etiqueta eje Y

# Hasta que no ejecutamos la siguiente línea no
# se muestra nada
plt.show()
```

Preste atención también a los comandos para especificar el título y las etiquetas de los ejes². Compruebe la documentación de estas funciones de [pyplot](#) para más posibilidades.

²En [python 2.x](#) se necesita preceder las tiras de caracteres con `u` (p. ej. `u'algunos números'`) para introducir una cadena de caracteres Unicode, con lo que se permiten acentos, eñes, etc.; en [python 3.x](#) no hace falta.



No hace falta que las abscisas y las ordenadas sean listas de números reales. Se pueden usar [arrays](#) de [numpy](#). Por ejemplo si las abscisas están en un array [xa](#) y las ordenadas en otro [ya](#), tendríamos: `plt.plot(xa, xb)`

Ejemplo 8.3.2. *Realicemos la representación gráfica de los puntos:*

(1, 1, 2.1), (2, 6), (3.3, 5.4), (4.9, 6.7),

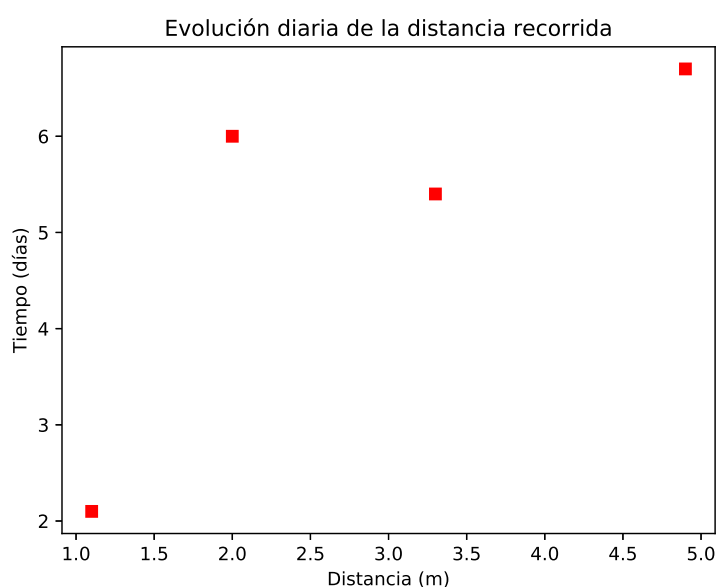
etiquetando el eje Ox con 'Distancia (m)' y el eje Oy con 'Tiempo (días)'. El título de la figura debe ser 'Evolución diaria de la distancia recorrida'.

Una solución ([plt2.py](#))

```
'''  
Ejemplo simple para pintar puntos con matplotlib  
'''  
import matplotlib.pyplot as plt  
  
puntos = [(1.1, 2.1),  
          (2, 6),  
          (3.3, 5.4),  
          (4.9, 6.7)]  
# Convertimos la lista de tuplas en 2 listas  
x, y = list(zip(*puntos))  
  
# Hacemos la gráfica, esta vez especificando un formato  
plt.plot(x, y, 'rs')
```

```
# Títulos y etiquetas
plt.xlabel('Distancia (m)')
plt.ylabel('Tiempo (días)')
plt.title('Evolución diaria de la distancia recorrida')

# La mostramos
plt.show()
```



8.3.4. Dibujo de funciones

No existe ninguna función específica en Matplotlib para dibujar una función matemática. Para hacerlo muestrearemos la función en la región en que queremos dibujarla calculando dos listas (o arrays) con las coordenadas x_i y las y_i de puntos de la curva. Si estos puntos están suficientemente cerca la representación será una muy buena aproximación de la curva continua.

Ejemplo 8.3.3. Representar las funciones $f(x) = x$ y $f(x) = x^3$ en el rango $[0, 5]$.

Una solución ([lineal_y_cubica.py](#))

```
'''
Ejemplo de representación de funciones; lineal y cúbica
'''
from numpy import *
import matplotlib.pyplot as plt
```

```
# Mallado en x relativamente compacto
x = arange(0, 5, 0.25)

# Calculamos  $x^3$  para los puntos anteriores
x3 = x**3

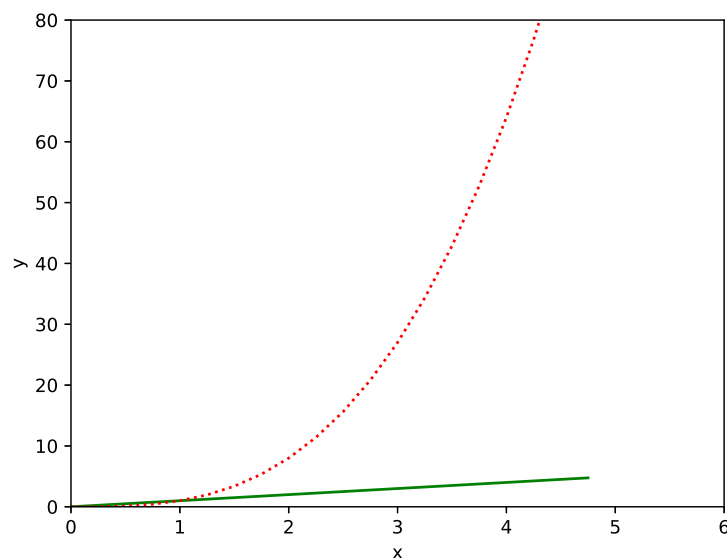
# Pintamos las dos gráficas
plt.plot(x, x, 'g-') # dibuja  $y = x$ 
plt.plot(x, x3, 'r:') # dibuja  $y = x^3$ 

# Etiquetas
plt.xlabel('x')
plt.ylabel('y')

# Especificamos límites a los ejes.
# Compruebe que pasa si no se especifica
plt.axis([0, 6, 0, 80])

# Mostramos la gráfica
plt.show()
```

El resultado es:



- `plt.plot(x, x, 'g-')` indica que queremos dibujar una curva con los puntos cuyas abscisas y ordenadas vienen dadas por `x`, con una línea continua (-) en color verde (green).

- `plt.plot(x, x3, 'r:')` y otra curva en color rojo (red) con una línea punteada (:) en la que las abscisas son `x` y las ordenadas los cubos de las abscisas.
- `plt.axis([0, 6, 0, 80])` indica que los ejes tienen los límites `xmin = 0`, `xmax = 6`, `ymin = 0`, `ymax = 80`.

8.3.5. Etiquetando las curvas

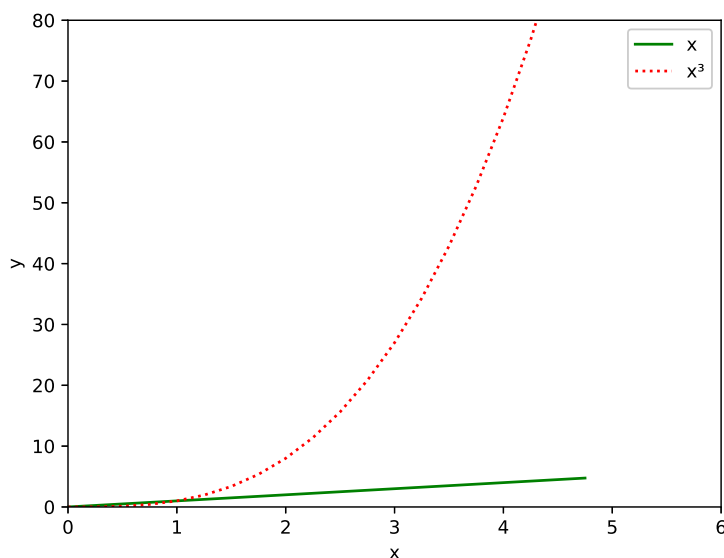
Se pueden incluir leyendas para etiquetar las curvas mediante la orden `legend()`. Para ello previamente hay que dibujar la curva incluyendo un argumento `label`:

```
plt.plot(x, x, 'g-', label='x')  
plt.plot(x, x3, 'r:', label='x³')
```

y luego usar:

```
plt.legend()
```

Si hacemos eso en el ejemplo anterior obtendremos una gráfica parecida a ésta:



Se pueden proporcionar también argumentos a `legend()`³ para, por ejemplo, indicar la posición de la leyenda o modificar el formato de presentación de la leyenda.

³Detalles de las distintas opciones en https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.legend.html

8.3.6. Control de los ejes

Los ejes se controlan con la función `axis()`, como antes hemos visto. Sin embargo se puede indicar o modificar con una cadena de caracteres algunas de las propiedades de los ejes⁴, como los límites o el tipo de escalado de los mismos. El concepto de *escalado* se refiere a la longitud real que ocupa en la pantalla o en una imagen una unidad. El escalado en el eje de ordenadas y en el eje de abscisas no tiene por qué ser el mismo.

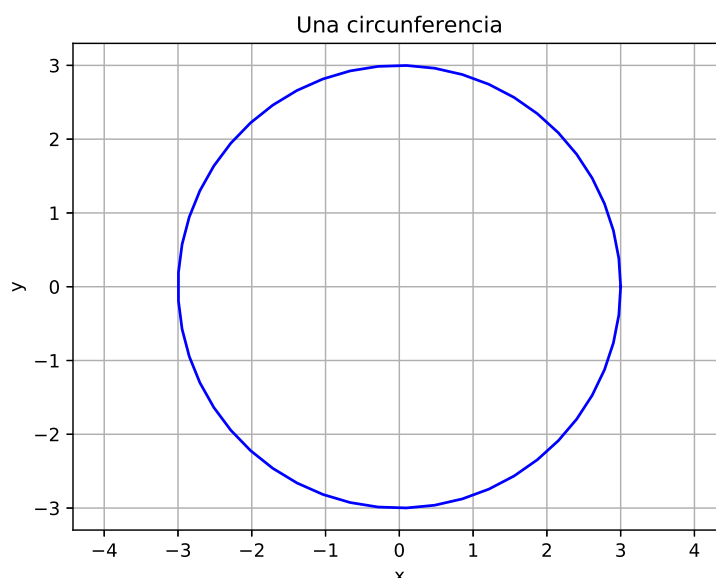
Por ejemplo, para hacer que el escalado sea idéntico en ambos se puede usar la opción `'equal'`. De esta manera las circunferencias parecen realmente circulares.

Ejemplo 8.3.4. *Un ejemplo con ejes iguales (`circunferencia.py`):*

```
'''  
Ejemplo de dibujo de una circunferencia de radio = 3  
usando coordenadas polares  
'''  
  
from numpy import *  
import matplotlib.pyplot as plt  
  
# Radio de la circunferencia (constante)  
r = 3.0  
  
# Muestreamos el ángulo polar entre 0 y 2*pi  
ang = linspace(0, 2*pi, 50)  
  
# Producimos la gráfica  
plt.plot(r*cos(ang), r*sin(ang), 'b-')  
  
# Etiquetas y título  
plt.xlabel('x')  
plt.ylabel('y')  
plt.title("Una circunferencia")  
  
# Hacemos que los ejes tengan un escalado idéntico  
plt.axis('equal')  
  
# Añadimos una rejilla para ver el efecto mejor.  
plt.grid()  
  
# Mostramos la figura  
plt.show()
```

⁴Detalles de las distintas opciones en https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.axis.html

Salida gráfica del ejemplo



Observe el efecto de la función `grid()`. Consulte la documentación y vea las opciones que admite (https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.grid.html).

Ejercicio 8.3.1. Realice una gráfica con las curvas de $\cos x$ y $\sin x$ entre 0 y 10 con 50 puntos. La primera debe ser en color azul con línea continua y la segunda color rojo y con rayas discontinuas. Etiquete la figura como 'Funciones trigonométricas' y con la leyenda correspondiente ('cos x', 'sin x') para las dos curvas.

Ejercicio 8.3.2. Diseñe una función denominada `funplot()` que dibuje una función de un parámetro `f` entre dos abscisas, `xmin`, `xmax` usando en total `np` puntos. También se le debe pasar el tipo de línea y la leyenda (que no dibujará si no se proporciona).

8.3.7. Guardando la figura

La función `savefig()` guarda la figura en un fichero gráfico para su posterior manipulación con otros programas.

Si le ponemos un nombre de fichero con una extensión como `emf`, `eps`, `pdf`, `png`, `jpg`, y otros, guardará el fichero con ese formato:

```
plt.savefig("velocidad.eps")
```

Consulte el manual (https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.savefig.html) para otras opciones.

8.3.8. Controlando propiedades de las líneas

Se pueden controlar las propiedades de las líneas usando la función `setp()`. Para ello debe guardarse el objeto producido por `plot()` mediante:

```
lines = plt.plot(x, x, 'bs', x, x3, 'r^')
```

De este modo, para cambiar el estilo de la línea se puede hacer:

```
plt.setp(lines, linestyle = ':')
```

(Véase el manual de referencia de Matplotlib para tener una lista de las propiedades modificables.)

Muchas de las opciones de las líneas o marcadores (anchura, tamaño, transparencia,...) pueden especificarse también directamente como parámetros de la función `plot()`.

8.3.9. Varias subfiguras

Se pueden dibujar en la misma figura varias gráficas (con diferentes ejes cada una) usando la función `subplot()`, que se invoca de la siguiente manera:

```
plt.subplot(numfilas, numcolumnas, numfigura)
```

Las figuras se numeran de izquierda a derecha y de arriba abajo. `plt.subplot(2, 4, ...)` sería la orden para dos filas y cuatro columnas, numeradas:

```
1  2  3  4
5  6  7  8
```

Ejemplo 8.3.5. *Un ejemplo con subfiguras* ([subfiguras.py](#))

```
'''
Ejemplo de creación de subfiguras en matplotlib
'''
from numpy import *
import matplotlib.pyplot as plt

# Creamos una función sencilla: oscilación amortiguada
def f(t): return exp(-t)*cos(2*pi*t)

# Muestreamos en tiempos con dos grosores:
t1 = arange(0.0, 5.0, 0.1) # Muestreado grueso
t2 = arange(0.0, 5.0, 0.02) # Muestreado fino

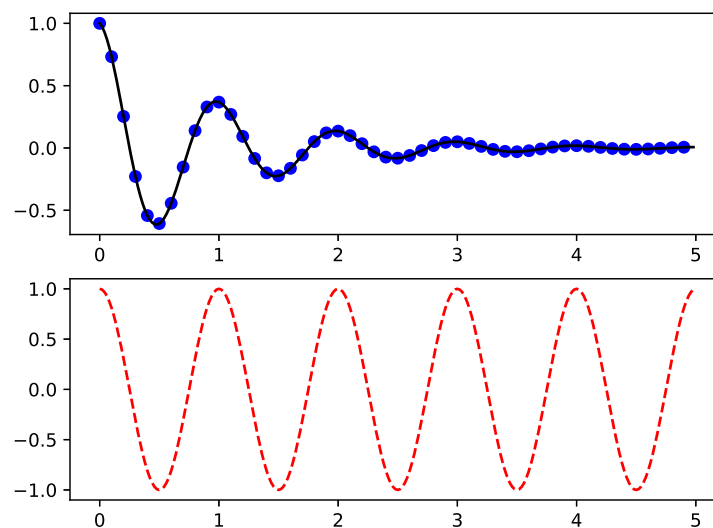
# Subfigura superior (2 filas, 1 columna, 1ª subfigura)
plt.subplot(2,1,1)
plt.plot(t1, f(t1), 'bo')
plt.plot(t2, f(t2), 'k')

# Subfigura inferior (2 filas, 1 columna, 2ª subfigura)
```

```
plt.subplot(2,1,2)
plt.plot(t2, cos(2*pi*t2), 'r--')

# Mostramos todo
plt.show()
```

Salida gráfica del ejemplo



8.3.10. Otro tipos de gráficas

`pyplot` tiene más variedad de funciones que permiten hacer otros tipos de representaciones gráficas. Algunas de las que están disponibles son:

- `hist()` para dibujar histogramas
- `bar()` para representar gráficas de barras verticales
- `errorbar()` para dibujar barras de error
- `semilogx()` para que el eje Ox esté en escala logarítmica
- `semilogy()` para que el eje Oy esté en escala logarítmica
- `loglog()` para que ambos ejes estén en esacala logarítmica

- `scatter()` para realizar diagramas de dispersión
- `polar()` para diagramas polares
- `contour()` para dibujar líneas de contorno (mapas de nivel)
- `contourf()` para dibujar rellenos de contorno
- `quiver()` para dibujar campos vectoriales

Ejemplo 8.3.6. Un ejemplo de `contour()` y `contourf()` (`contorno_gaussianas.py`)

```
'''
Ejemplo de cómo dibujar curvas de nivel o contornos que permiten
representar superficies tridimensionales sobre un papel.
Se representan la superposición de 2 gaussianas 2D
perpendiculares entre sí.
'''
import matplotlib.pyplot as plt
from numpy import mgrid, exp

# Creamos una cuadrícula (mallado) en el plano XY
x, y = mgrid[-1:1:0.01, -1:1:0.01]

# Calculamos las alturas para esos valores
z = exp(-(4*x**2 + y**2/32)) + exp(-(x**2/32 + 4*y**2))

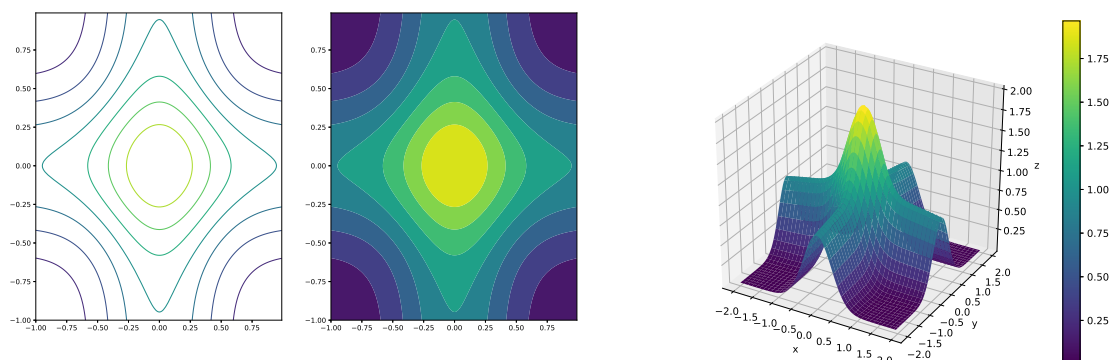
# A por la figura
plt.figure(figsize=(14,8))

# A la izquierda representamos solamente las curvas de nivel
plt.subplot(1,2,1)
plt.contour(x, y, z)

# A la derecha rellenos con colores los espacios entre las curvas
plt.subplot(1,2,2)
plt.contourf(x, y, z)

# Y finalmente, mostramos la figura
plt.show()
```

Salida gráfica del ejemplo



A la izquierda se puede ver el resultado de ejecutar el código del ejemplo. A la derecha se muestra en 3D la superficie cuyas curvas se están representando.

Ejemplo 8.3.7. La función $y = ax^b$ se representa mejor con escalas logarítmicas en ambos ejes, usando `loglog` (realmente sale una recta).

Realice la representación gráfica de la función anterior con $a = 10.0$, $b = 0.3$, entre 10^{-3} y 10 usando 50 puntos.

Una solución (`logloggraf.py`)

```
'''
Ejemplo de uso de la función loglog() para representar
la función  $y = a \cdot x^b$ 
'''
from numpy import *
import matplotlib.pyplot as plt

a, b = 10.0, 0.3 # Coeficientes

# Usamos un malla logarítmico de 51 puntos entre
# 1E-7 y 1E7
x = logspace(-7, 7, 51)

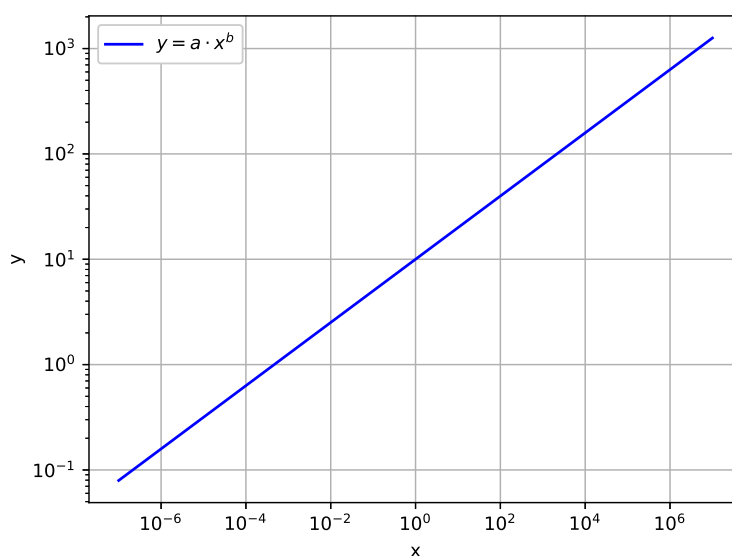
y = a*x**b # Calculamos los puntos

# Representación en escala logarítmica para ambos ejes
plt.loglog(x, y, 'b-', label='$y = a \cdot x^b$')

# Añadimos una rejilla, la leyenda y los ejes
plt.grid()
plt.legend()
plt.xlabel('x')
plt.ylabel('y')
```

```
plt.show()
```

Salida gráfica del ejercicio



8.3.11. Anotando figuras

Se puede incluir una cadena de caracteres en un punto (xt, yt) mediante la función `text()`:

```
plt.text(xt, yt, "Texto")
```

Se puede añadir una flecha usando `annotate()`:

```
plt.annotate("Texto", xy = (xa, ya), xytext = (xt, yt),  
            arrowprops=dict(facecolor='black', shrink=0.05, width=1))
```

que coloca el texto en las coordenadas (xt, yt) , con una flecha que tiene su punta en (xa, ya) .

Ejemplo 8.3.8. Dibuje la función $y = 0.2 \cdot e^{-(x+3)^2}$ entre las abscisas -5 y 0 , y las ordenadas 0 y 0.3 , etiquetando el máximo de la función con 'máximo' y una flecha indicativa.

Una solución ([text_annotate.py](#))

```
'''
Ejemplo del uso de text() y annotate() en matplotlib para
añadir texto en las gráficas
'''
import matplotlib.pyplot as plt
from numpy import *

# Parámetros del problema: Intervalo y número de puntos
xmin, xmax, ymin, ymax, np = -5.0, 0.0, 0.0, 0.3, 51

# Construimos la función
x = linspace(xmin, xmax, np)
y = 0.2*exp(-(x+3.0)**2)

# Dibujamos la función
plt.plot(x, y, 'b-')

# Etiquetamos los ejes
plt.xlabel('x');
plt.ylabel('y')

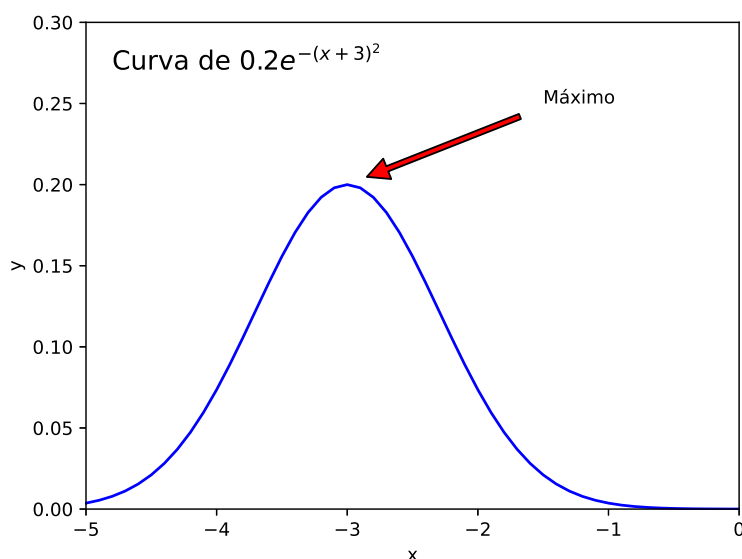
# Restringimos los ejes
plt.axis([xmin, xmax, ymin, ymax])

# Ponemos un texto simple
plt.text(-4.8, 0.27, 'Curva de $0.2 e^{-(x+3)^2}$', fontsize='x-large')

# Ponemos un texto con una flecha
plt.annotate("Máximo", xy = (-3.0, 0.2), xytext = (-1.5, 0.25),
            arrowprops=dict(facecolor='red', shrink=0.1, width=3))

# Mostramos la gráfica
plt.show()
```

Salida gráfica del ejercicio



8.4. Gráficas en 3D

Para visualización de curvas y superficies en 3D usaremos el módulo `matplotlib.mplot3d`.

Evidentemente no existe posibilidad física de representar en 3D sino que se recurre a proyecciones sobre la pantalla (2D).

Hay más información disponible en <https://matplotlib.org/stable/api/toolkits/mplot3d.html>

8.4.1. Usando `mplot3d`

Para usar ese módulo se debe usar además el submódulo `Axes3D` de la manera siguiente:

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
```

Luego se asocia la figura que se crea con el comando `Axes3D` a unos ejes, `ax`, en 3D:

```
fig = plt.figure()
ax = Axes3D(fig)
```

8.4.2. Dibujo de curvas con `mplot3d`

Ahora ya se pueden dibujar curvas proyectadas en la pantalla mediante:

```
ax.plot(x, y, z, label='una curva')
plt.show()
```

(obviamente después de definir los vectores `x`, `y`, `z`)

Algunas de las funciones (como `legend()`) que usábamos en las graficas 2D se pueden usar también ahora, pero a través del objeto `ax`. Por ejemplo para producir una leyenda tendríamos que incluir `ax.legend()`. Sin embargo para etiquetar los ejes debemos usar las funciones `set_xlabel()`, `set_ylabel()` y `set_zlabel()` definidas en el módulo `Axes3D` (y por tanto precedidas por `ax.`).

Ejemplo 8.4.1. *Un ejemplo de curva en 3D* (`plot3d.py`)

```
'''
Ejemplo de representación de una espiral en 3D
'''

# Importamos el paquete Axes3D
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt
from numpy import *

# Producimos la curva
th = linspace(-4*pi, 4*pi, 200)
z = linspace(-2, 2, 200)
r = z**2 + 1; x = r*sin(th); y = r*cos(th)

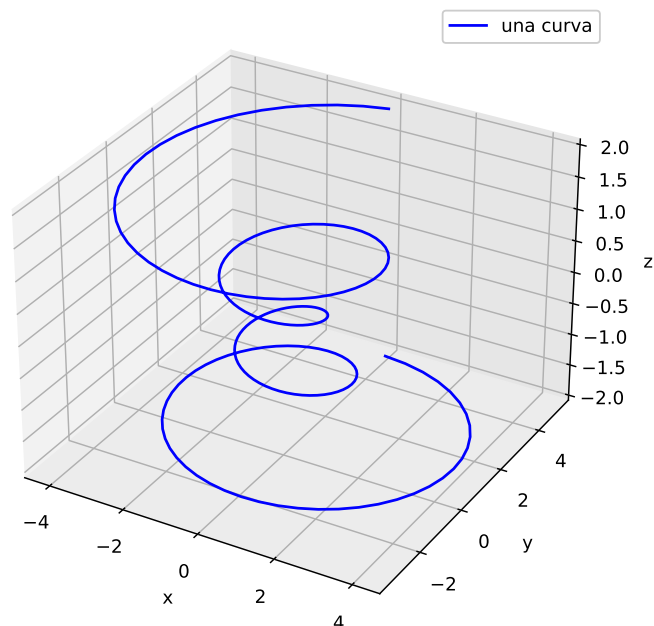
# Creamos la figura y los ejes 3D
fig = plt.figure()
ax = Axes3D(fig)

# Dibujamos la curva
ax.plot(x, y, z, 'b-', label='una curva')

# Añadimos la leyenda y las etiquetas de los ejes
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("z")
ax.legend()

# Mostramos la curva
plt.show()
```

Salida gráfica del ejemplo

8.4.3. Superficies con `mplot3d`

También se pueden dibujar superficies proyectadas en la pantalla mediante `ax.plot_surface(x, y, z)`.

El mapa de colores que se asocian a los valores de z se puede variar, usando por ejemplo los suministrados en el módulo `cm` que se importa mediante

```
from matplotlib import cm
```

Se puede consultar el listado completo de mapas de color implementados en matplotlib en https://matplotlib.org/stable/gallery/color/colormap_reference.html.

Ejemplo 8.4.2. Un ejemplo (`superficie.py`) de representación de una superficie en 3D definida por la ecuación $z = \frac{3}{4}y(3x^2 - y^2)$

```
'''
Ejemplo de representación de una superficie en 3D definida por
z = 3*y*(3*x**2-y**2)/4
'''
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt
from matplotlib import cm
from numpy import *
```

```
# Creamos un marray homogéneo en el plano XY
x, y = mgrid[-1:1:0.05, -1:1:0.05]

# Calculamos las coordenada de la superficie
# en cada uno de los puntos (x,y)
z = 3*y*(3*x**2-y**2)/4

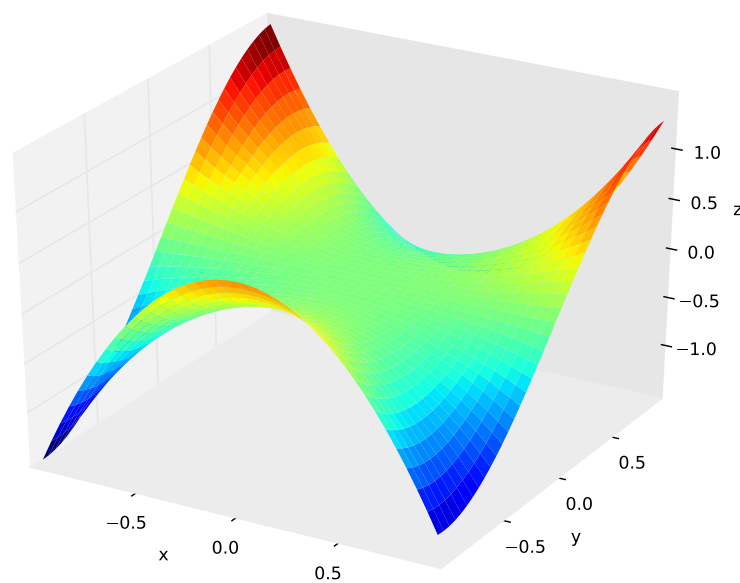
# Creamos la figura y los ejes 3D
fig = plt.figure()
ax = Axes3D(fig)

# Pintamos la superficie
ax.plot_surface(x, y, z, rstride=1, cstride=1, cmap=cm.jet)

# Etiquetamos los ejes
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("z")

# Mostramos la figura
plt.show()
```

Salida gráfica del ejemplo



Ejemplo 8.4.3. *Dibuje la función*

$$z = 0.2 \sin(x) e^{-(x+3)^2} e^{-(y+2)^2}$$

con $x \in [-0, 0]$, $y \in [-5, 0]$.

Una solución ([superficie2.py](#))

```
'''
Ejemplo de representación de una superficie en 3D definida por
z = 0.2*sin(x)*exp(-(x+3)**2)*exp(-(y+2)**2)
'''
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt
from matplotlib import cm
from numpy import *

# Creamos un mado homogéneo en el plano XY (forma alternativa)
xv = yv = arange(-5, 0, 0.1)
x, y = meshgrid(xv, yv);

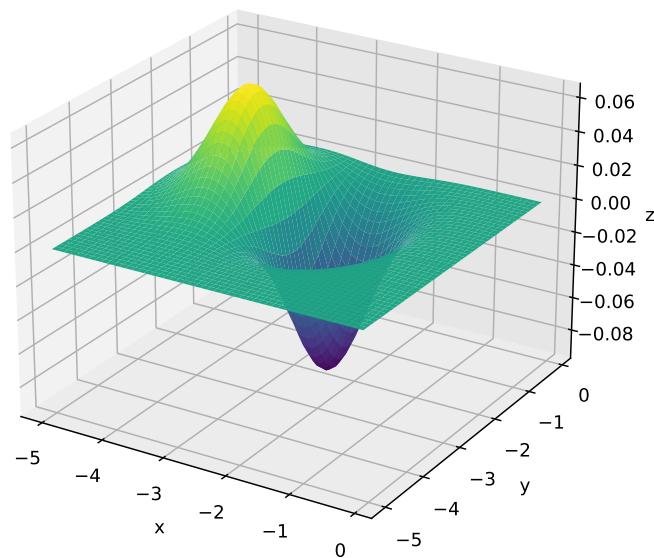
# Calculamos las coordenada de la superficie
# en cada uno de los puntos (x,y)
z = 0.2*sin(x)*exp(-(x+3)**2)*exp(-(y+2)**2)

# Creamos la figura y los ejes 3D
fig = plt.figure()
ax = Axes3D(fig)

# Pintamos la superficie
ax.plot_surface(x, y, z, rstride=1, cstride=1, cmap=cm.viridis)

# Etiquetamos los ejes
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("z")

# Mostramos la figura
plt.show()
```

Salida gráfica del ejercicio

Ejercicio 8.4.1. Represente la superficie del Ejemplo 8.3.6. Observe que la figura mostrada en el ejemplo incluye la barra con el código de colores.